RecoveryOps: Branch-Level Database Schema

1. Executive Summary & Architecture Scope

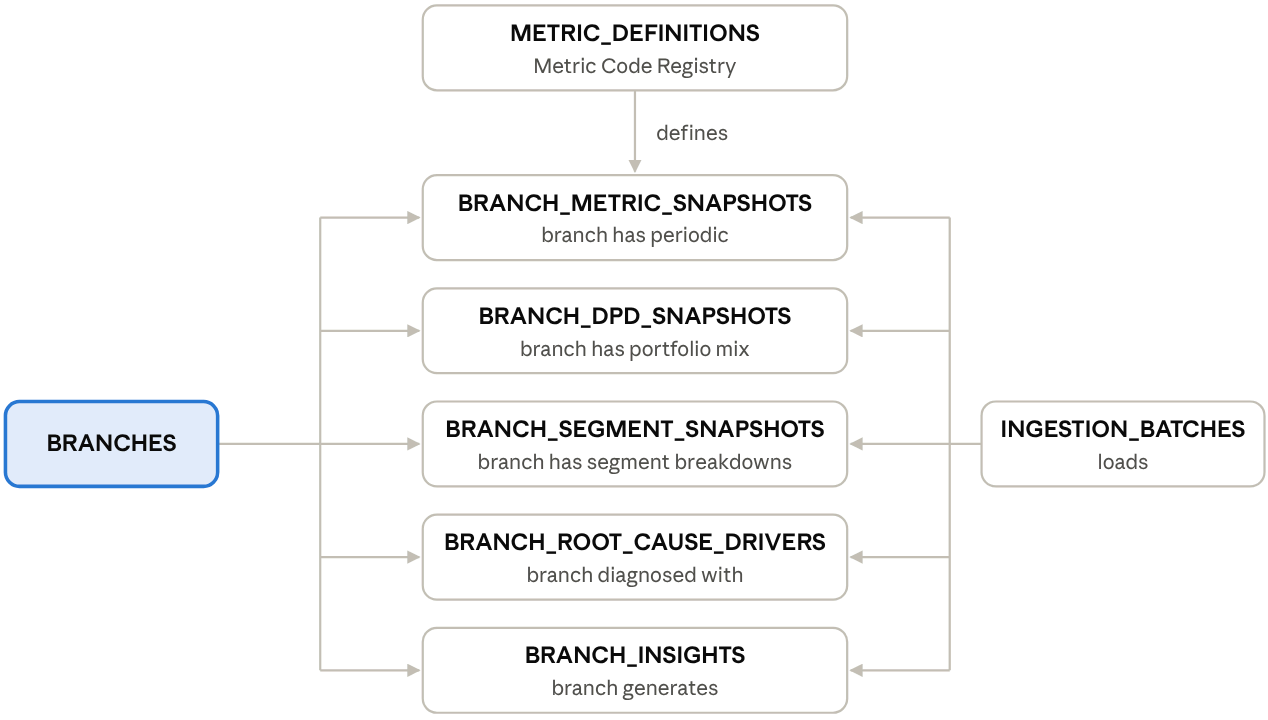
Frontend Route	Screen Title	Purpose & Visual Components	Primary Backing Tables
(State A)	**Branch Network Analytics**	Network ranking table, regional averages (Southeast, Midwest, etc.), top performer vs. lagging branch callouts across 5 core KPIs.	`branches`, `branch_metric_snapshots`
(State B)	**Selected Branch Overview**	Single-branch command center, net performance gap hero banner (+/- pts vs peer avg), 5 core KPI cards, prescribed outreach sequence, primary focus diagnostic.	`branches`, `branch_metric_snapshots`, `branch_insights`
`/performance`	**Performance & Portfolio Mix**	Historical collection trend chart (Aug–Sep vs peer baseline), 5-bucket DPD portfolio distribution pie chart, 5 core KPI month-over-month performance bars.	`branch_metric_snapshots`, `branch_dpd_snapshots`

Frontend Route	Screen Title	Purpose & Visual Components	Primary Backing Tables
`/root-cause`	**Strategic Decision Engine**	Head-to-head peer benchmarking, 5 core KPI variance decomposition, segment cohort breakdowns (Product, Window, Horizon), tactical playbooks, projected metric lift.	`branch_root_cause_drivers`, `branch_segment_snapshots`, `branch_insights`

---

2. Branch-Level Entity-Relationship Architecture

Branches and ingestion batches feed five branch-level tables



entity relationships · 8 tables

---

3. Database Schema Specification (Strictly Branch Level)

Table 1: `branches` (Organizational Hierarchy)

Master registry of all regional branch offices evaluated in network rankings and peer comparisons.

```
CREATE TABLE branches (
    id SERIAL PRIMARY KEY,
    source_branch_code VARCHAR(16) UNIQUE NOT NULL, -- e.g., '101', '102',
'201'
    name VARCHAR(100) NOT NULL,                    -- e.g., 'Tampa Branch'
    region VARCHAR(50)
    state VARCHAR(10),                             -- e.g., 'FL', 'TX', 'GA'
    active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_branches_region ON branches (region);
CREATE INDEX idx_branches_active ON branches (active);
```

Table 2: `ingestion_batches` (Data Lineage & Batch Metadata)

Tracks periodic report ingestion batches and distinguishes demo/synthetic runs from production runs.

```
CREATE TABLE ingestion_batches (
    id SERIAL PRIMARY KEY,
    source VARCHAR(64) NOT NULL,                    -- e.g.,
'report_ingestion', 'synthetic_demo'
    report_type VARCHAR(64),                        -- e.g.,
'branch_delinquency_dashboard'
    as_of_date DATE NOT NULL,                       -- Global business date
snapshot (e.g., '2026-09-15')
    period_start DATE,                              -- Reporting window start
(e.g., '2026-09-01')
    period_end DATE,                                -- Reporting window end
(e.g., '2026-09-30')
    is_demo BOOLEAN NOT NULL DEFAULT FALSE,
    notes TEXT,
    loaded_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_ingestion_batches_date ON ingestion_batches (as_of_date);
CREATE INDEX idx_ingestion_batches_demo ON ingestion_batches (is_demo);
```

Table 3: `metric_definitions` (Metric Code Registry)

Standardized metric metadata registry ensuring parity between frontend metric codes and backend calculations.

```
CREATE TABLE metric_definitions (
    id SERIAL PRIMARY KEY,
    metric_code VARCHAR(64) UNIQUE NOT NULL,        -- e.g.,
'collection_rate', 'avg_resolution_days'
    name VARCHAR(100) NOT NULL,                     -- e.g., 'Collection
Efficiency %'
    unit VARCHAR(16) NOT NULL CHECK (unit IN ('percent', 'currency', 'count',
'days', 'ratio')),
    description TEXT,
    has_numerator_denominator BOOLEAN NOT NULL DEFAULT FALSE,
    version INT NOT NULL DEFAULT 1,
    effective_from DATE
);
```

Table 4: `branch_metric_snapshots` (Periodic Branch Performance)

Stores historical and current metric values per branch per period. Directly powers `/` (Rankings & Overview) and `/performance` (Trends & KPI Analytics).

```
CREATE TABLE branch_metric_snapshots (
    id SERIAL PRIMARY KEY,
    branch_id INT NOT NULL REFERENCES branches(id) ON DELETE CASCADE,
    batch_id INT NOT NULL REFERENCES ingestion_batches(id) ON DELETE CASCADE,
    metric_code VARCHAR(64) NOT NULL,              -- e.g.,
'collection_rate', 'contact_rate'
    value NUMERIC(18,6),                           -- Calculated numeric
value (e.g., 71.0 or 27.0)
    numerator BIGINT,                              -- e.g., Recovered amount
or contacted accounts
    denominator BIGINT,                            -- e.g., Overdue balance
or total delinquent accounts
    unit VARCHAR(16) NOT NULL CHECK (unit IN ('percent', 'currency', 'count',
'days', 'ratio')),
    availability VARCHAR(16) NOT NULL DEFAULT 'available' CHECK (availability
IN ('available', 'unavailable', 'stale')),
    period_start DATE,                             -- Monthly period start
(e.g., '2026-09-01')
    period_end DATE,                               -- Monthly period end
(e.g., '2026-09-30')
    as_of_date DATE NOT NULL,                      -- Snapshot calculation
date
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_branch_metric UNIQUE (branch_id, metric_code, period_end,
as_of_date)
);

CREATE INDEX idx_bms_branch_metric ON branch_metric_snapshots (branch_id,
metric_code, as_of_date);
CREATE INDEX idx_bms_period ON branch_metric_snapshots (period_end,
metric_code);
```

Table 5: `branch_dpd_snapshots` (DPD Portfolio Mix)

Stores the 5-bucket delinquency distribution per branch. Powers the portfolio mix chart and pie chart on `/performance` and `/` overview.

```
CREATE TABLE branch_dpd_snapshots (
    id SERIAL PRIMARY KEY,
    branch_id INT NOT NULL REFERENCES branches(id) ON DELETE CASCADE,
    batch_id INT NOT NULL REFERENCES ingestion_batches(id) ON DELETE CASCADE,
    bucket VARCHAR(16) NOT NULL CHECK (bucket IN ('Current', '1-30', '31-60',
'61-90', '90+')),
    account_count INT NOT NULL CHECK (account_count >= 0),
    balance NUMERIC(14,2),                          -- Total outstanding
balance in bucket
    portfolio_share NUMERIC(6,4),                   -- Fractional share of
total portfolio (e.g., 0.3400 = 34%)
    as_of_date DATE NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_branch_dpd UNIQUE (branch_id, bucket, as_of_date)
);

CREATE INDEX idx_bds_branch_date ON branch_dpd_snapshots (branch_id,
as_of_date);
```

Table 6: `branch_segment_snapshots` (Diagnostic Sub-Segments) - **OPTIONAL**

Stores granular operational slices per branch (e.g., Calling Window, Loan Product, Promise Horizon) used in diagnostic gap breakdowns on `/root-cause`.

```
CREATE TABLE branch_segment_snapshots (
    id SERIAL PRIMARY KEY,
    branch_id INT NOT NULL REFERENCES branches(id) ON DELETE CASCADE,
    batch_id INT NOT NULL REFERENCES ingestion_batches(id) ON DELETE CASCADE,
    segment_type VARCHAR(50) NOT NULL,             -- e.g., 'product_type',
'calling_window', 'ptp_horizon'
    segment_value VARCHAR(100) NOT NULL,           -- e.g., 'Retail Personal
Loans', 'Evening Window'
    metric_code VARCHAR(64) NOT NULL,              -- e.g.,
'collection_rate', 'contact_rate'
    value NUMERIC(18,6),                           -- Branch segment value
    account_count INT,                             -- Account volume in
segment
    as_of_date DATE NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_branch_segment UNIQUE (branch_id, segment_type,
segment_value, metric_code, as_of_date)
);

CREATE INDEX idx_bss_branch_segment ON branch_segment_snapshots (branch_id,
segment_type, as_of_date);
```

Table 7: `branch_root_cause_drivers` (Strategic Decision Engine)

Pre-computed root cause gap diagnostics, peer comparisons, tactical playbooks, and projected payoffs for the 5 core KPIs on `/root-cause`.

```
CREATE TABLE branch_root_cause_drivers (
    id SERIAL PRIMARY KEY,
    branch_id INT NOT NULL REFERENCES branches(id) ON DELETE CASCADE,
    as_of_date DATE NOT NULL,
    driver_id VARCHAR(32) NOT NULL CHECK (driver_id IN (
        'collection_efficiency', 'avg_resolution_days', 'ptp_kept',
'contact_rate', 'dpd_distribution'
    )),
    branch_value NUMERIC(10,2) NOT NULL,           -- Branch actual metric
value
    peer_value NUMERIC(10,2) NOT NULL,             -- Regional cohort
benchmark value
    delta NUMERIC(10,2) NOT NULL,                  -- Net gap: branch_value -
peer_value
    direction VARCHAR(32) NOT NULL CHECK (direction IN ('Lead Advantage',
```

```
'Underperforming', 'Elevated Risk', 'Optimal Pace')),
    variance_rationale TEXT NOT NULL,              -- Quantitative variance
description
    attribution_rationale TEXT NOT NULL,           -- Operational frontline
root cause
    key_takeaway TEXT NOT NULL,                    -- Executive leadership
summary
    one_liner_insights JSONB NOT NULL DEFAULT '[]'::jsonb, -- Array of [{
iconType, label, text }]
    segments JSONB NOT NULL DEFAULT '[]'::jsonb,           -- Cohort share
comparison table
    interventions JSONB NOT NULL DEFAULT '[]'::jsonb,      -- Actionable
frontline playbooks
    metric_lift JSONB NOT NULL DEFAULT '[]'::jsonb,        -- Expected metric
impact tags
    projected_lift VARCHAR(50) NOT NULL,           -- e.g., '+5.5–8.0 pts'
    projected_payoff VARCHAR(50) NOT NULL,         -- e.g., '+$1.4M Net
Recovery'
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_branch_rc_driver UNIQUE (branch_id, driver_id, as_of_date)
);

CREATE INDEX idx_brcd_branch_date ON branch_root_cause_drivers (branch_id,
as_of_date);
```

Table 8: `branch_insights` (AI Diagnostics & Evidence)

Generative and rule-based diagnostic narratives with structured metric evidence items.

```
CREATE TABLE branch_insights (
    id SERIAL PRIMARY KEY,
    branch_id INT NOT NULL REFERENCES branches(id) ON DELETE CASCADE,
    batch_id INT NOT NULL REFERENCES ingestion_batches(id) ON DELETE CASCADE,
    narrative TEXT NOT NULL,                        -- Executive diagnosis
narrative
    evidence JSONB NOT NULL DEFAULT '[]'::jsonb,   -- Array of [{ metricCode,
branchValue, networkValue, delta, unit }]
    model_version VARCHAR(32),
    generated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    as_of_date DATE NOT NULL
);

CREATE INDEX idx_insights_branch_date ON branch_insights (branch_id,
as_of_date);
```

---

4. Branch Metric Calculation Engine & Formulas

Here are the calculation formulas, component definitions, and verified SQL queries for all branch-level metrics.

4.1. The 5 Core Branch KPIs

1. Collection Efficiency % (Collection Rate)

- **Frontend Surface**: Network Rankings Table, Branch Hero Gap Card, 6-Month Trend Chart, Root Cause Driver #1.
- **Calculation Formula**:

$$\text{Collection Efficiency (\%)} = \frac{\text{Total Cash Payments Collected in Period}}{\text{Total Overdue Balance at Period Start}} \times 100$$

  - **Numerator**: Total cleared payments received during the billing period.
  - **Denominator**: Total past-due delinquent balance at period start.
  - **Example**: If a branch collects **$4.8M** against **$6.0M** overdue at month start: (4,800,000 / 6,000,000) × 100 = 80.0%
- **SQL Query (from Snapshots)**:

```
SELECT
    branch_id,
    ROUND(value, 1) AS collection_efficiency_pct
FROM branch_metric_snapshots
WHERE metric_code = 'collection_rate'
  AND as_of_date = '2026-09-15';
```

---

2. Average Days to Resolution (Velocity in Days)

- **Frontend Surface**: Overview KPI Card, Performance Analytics, Root Cause Driver #2.
- **Calculation Formula**:

```
                                Sum of Resolution Durations for All
Resolved Accounts
Avg Days to Resolve (days) =
_____________________________________________________________________

                                            Total Number of Resolved
Accounts
```

  - **Duration per Account**: `Resolution Date (Date of Settlement/Payment)` minus `Delinquency Onset Date (Due Date)`.
  - **Target Benchmark**: Peer network average is **22 days**. Lower duration preserves borrower liquidity and prevents accounts rolling forward.
- **SQL Query**:

```
SELECT
    branch_id,
    ROUND(value, 0) AS avg_days_to_resolve
FROM branch_metric_snapshots
WHERE metric_code = 'avg_days_to_resolve'
  AND as_of_date = '2026-09-15';
```

---

3. Contact Rate %

- **Frontend Surface**: Network Directory, Branch KPI Card, Performance Analytics, Root Cause Driver #4.
- **Calculation Formula**:

$$\text{Contact Rate (\%)} = \frac{\text{Delinquent Accounts with } \geq 1 \text{ Connected Contact}}{\text{Total Delinquent Accounts in Branch}} \times 100$$

  - **Numerator**: Distinct overdue accounts with at least one verified two-way conversation (`Connected` or `Promise to Pay`).
  - **Denominator**: Total overdue accounts (`DPD > 0`) in the branch portfolio.
- **SQL Query**:

```
SELECT
    branch_id,
    ROUND(value, 1) AS contact_rate_pct
FROM branch_metric_snapshots
WHERE metric_code = 'contact_rate'
  AND as_of_date = '2026-09-15';
```

---

4. Promise to Pay (PTP) Kept Rate %

- **Frontend Surface**: Network Rankings Table, Performance Analytics, Root Cause Driver #3.
- **Calculation Formula**:

$$\text{PTP Kept Rate (\%)} = \frac{\text{Promises Honored with Cleared Payment (status = 'kept')}}{\text{Total Payment Promises Due in Period}} \times 100$$

  - **Numerator**: Number of commitments fulfilled where payment cleared on or before the commitment date.
  - **Denominator**: Total commitments with promised date falling within the evaluation window.
- **SQL Query**:

```
SELECT
    branch_id,
    ROUND(value, 1) AS ptp_kept_pct
FROM branch_metric_snapshots
WHERE metric_code = 'ptp_kept_rate'
  AND as_of_date = '2026-09-15';
```

---

5. DPD Delinquency Concentration %

- **Frontend Surface**: Ranking Table, Pie Chart, Root Cause Driver #5.
- **Calculation Formula**:

$$\text{30+ DPD Concentration (\%)} = \frac{\text{Accounts with DPD} \geq \text{31 Days}}{\text{Total Active Accounts in Branch}} \times 100$$

  - **Definition**: Proportion of portfolio accounts that have slipped past 30 days past due. High concentration creates credit loss provision drag.
- **SQL Query**:

```
SELECT
    branch_id,
    ROUND(value * 100, 1) AS dpd_30_plus_pct
FROM branch_metric_snapshots
WHERE metric_code = 'delinquency_rate_30_plus'
  AND as_of_date = '2026-09-15';
```

4.2. Portfolio Benchmarking & Cohort Metrics

6. Net Performance Gap vs. Peer Benchmark (+/- Points)

- **Frontend Surface**: Command Hero Banner on `/` (State B) and `/root-cause`.
- **Calculation Formula**:

```
Net Performance Gap (pts) = Branch Collection Rate − Network Peer Average
Collection Rate
```

  - **Positive Gap (`+` pts)**: Branch is outperforming peers; tone is **success** (`Above Peer Benchmark`).

  - **Negative Gap (`-` pts)**: Branch is lagging peers; tone is **danger** (`Lagging Peer Cohort`).
- **SQL Query**:

```
WITH network_benchmark AS (
    SELECT AVG(value) AS peer_avg_rate
    FROM branch_metric_snapshots
    WHERE metric_code = 'collection_rate'
      AND as_of_date = '2026-09-15'
)
SELECT
    b.id AS branch_id,
    b.name AS branch_name,
    ROUND(s.value, 1) AS branch_rate,
    ROUND(nb.peer_avg_rate, 1) AS peer_avg_rate,
    ROUND(s.value - nb.peer_avg_rate, 1) AS net_gap_pts,
    CASE
        WHEN s.value >= nb.peer_avg_rate THEN 'Above Peer Benchmark'
        ELSE 'Lagging Peer Cohort'
    END AS status_banner
FROM branch_metric_snapshots s
JOIN branches b ON b.id = s.branch_id
CROSS JOIN network_benchmark nb
WHERE s.metric_code = 'collection_rate'
  AND s.as_of_date = '2026-09-15';
```

---

7. DPD Bucket Portfolio Share % (5-Stage Mix)

- **Frontend Surface**: DPD Pie Chart and Progress Bar on `/performance` .
- **Buckets**: `Current` , `1-29` , `30-59` , `60-89` , `90+` .
- **Calculation Formula**:

$$\text{Portfolio Share (\%)} = \frac{\text{Account Count in Bucket}}{\text{Total Accounts in Branch}} \times 100$$

- **SQL Query**:

```
SELECT
    branch_id,
    bucket,
    account_count,
    balance,
    ROUND(portfolio_share * 100, 1) AS portfolio_share_pct
FROM branch_dpd_snapshots
WHERE as_of_date = '2026-09-15'
ORDER BY
    CASE bucket
        WHEN 'Current' THEN 1
        WHEN '1-29' THEN 2
        WHEN '30-59' THEN 3
        WHEN '60-89' THEN 4
        WHEN '90+' THEN 5
    END;
```

---

8. Sub-Segment Cohort Contribution Delta (+/- Points)

- **Frontend Surface**: Cohort Breakdown Tables on `/root-cause`.
- **Calculation Formula**:

```
Segment Delta (pts) = Branch Segment Share (%) - Network Peer Segment
Share (%)
```

- **SQL Query**:

```
WITH network_segments AS (
    SELECT
        segment_type,
        segment_value,
        AVG(value) AS network_avg_share
    FROM branch_segment_snapshots
    WHERE as_of_date = '2026-09-15'
      AND metric_code = 'collection_rate'
    GROUP BY segment_type, segment_value
)
SELECT
    s.branch_id,
    s.segment_type,
    s.segment_value,
    ROUND(s.value, 1) AS branch_share_pct,
```

```
    ROUND(ns.network_avg_share, 1) AS network_share_pct,
    ROUND(s.value - ns.network_avg_share, 1) AS delta_pts,
    s.account_count
FROM branch_segment_snapshots s
JOIN network_segments ns
  ON ns.segment_type = s.segment_type
 AND ns.segment_value = s.segment_value
WHERE s.as_of_date = '2026-09-15'
ORDER BY ABS(s.value - ns.network_avg_share) DESC;
```
